

Claude Certified Architect - Learning Cards

Flashcards for exam preparation organized by domain with visual illustrations

How to Use These Cards

- **Question:** Read the question and test your knowledge
 - **Answer:** Check your understanding against the detailed answer
 - **Mermaid Diagrams:** Visual illustrations help reinforce the concepts
 - **Key Concepts:** Highlighted boxes show critical principles and anti-patterns
-

Domain 1: Agentic Architecture & Orchestration (27%)

Card 1.1: Agentic Loop Control

Question

What determines when to continue vs stop an agentic loop?

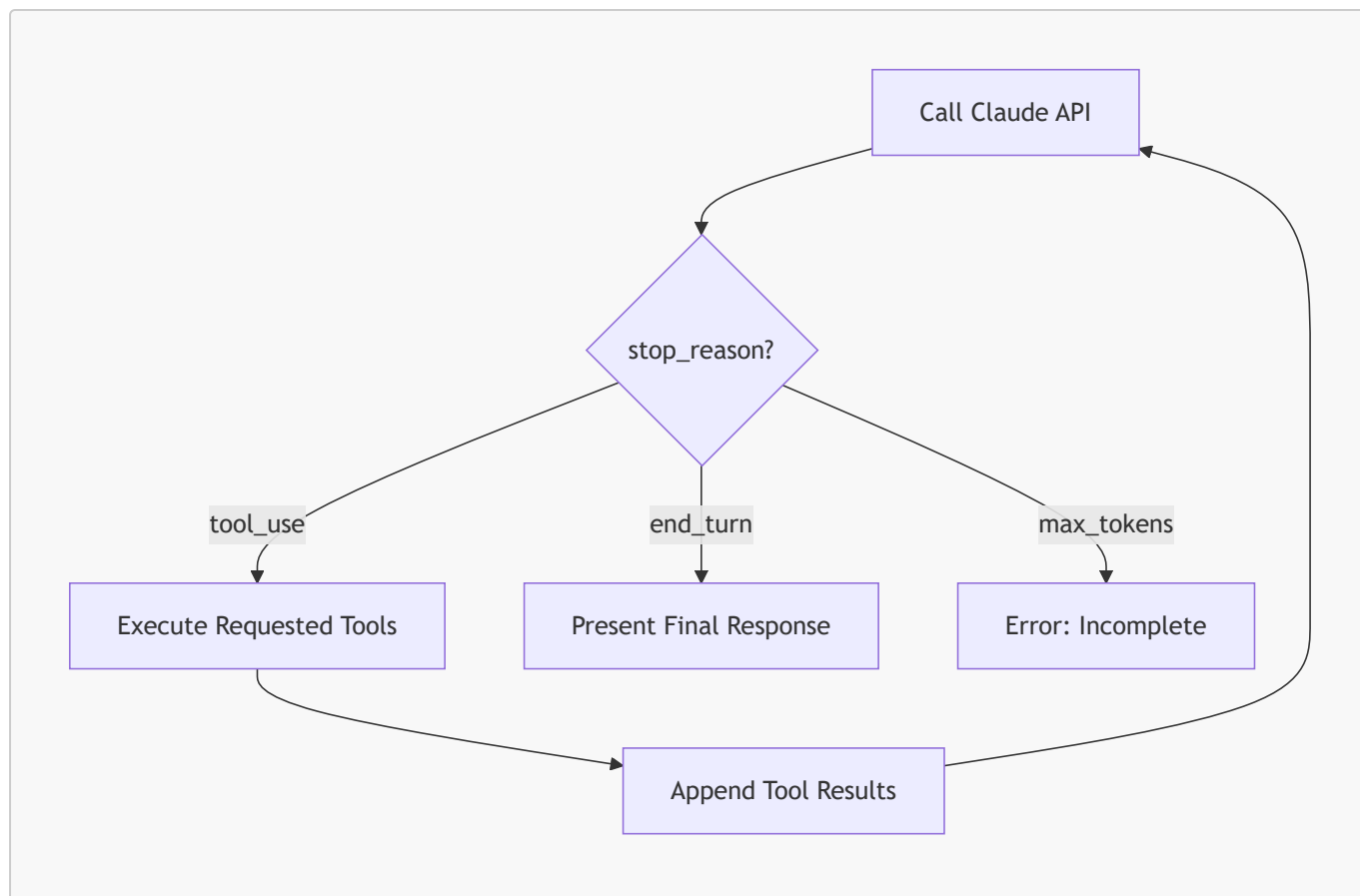
Answer

Check the `stop_reason` field:

- Continue when `stop_reason === "tool_use"`
- Stop when `stop_reason === "end_turn"`

Key Concept: Anti-patterns to Avoid

Avoid parsing natural language signals, using arbitrary iteration caps as primary stopping mechanism, or checking assistant text content for completion indicators.



Card 1.2: Programmatic Prerequisites

Question

How do you implement a programmatic prerequisite for critical business rules?

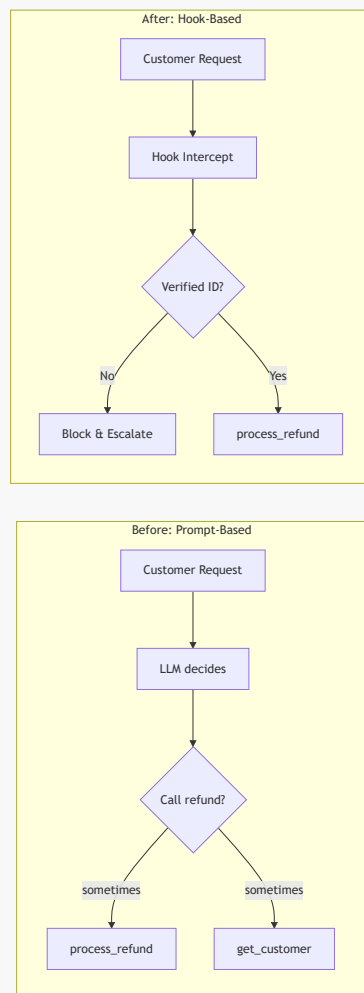
Answer

Use hooks to enforce deterministic compliance:

- Block downstream tool calls until prerequisites complete (e.g., block `process_refund` until `get_customer` returns verified ID)
- Implement `PostToolUse` hooks for data normalization
- Intercept tool calls to block policy violations (e.g., refunds above \$500)

Key Concept

Prompt-based enforcement has non-zero failure rate. Use hooks when deterministic compliance is required for critical operations.



Card 1.3: Hub-and-Spoke Architecture

Question

What is the hub-and-spoke architecture in multi-agent systems?

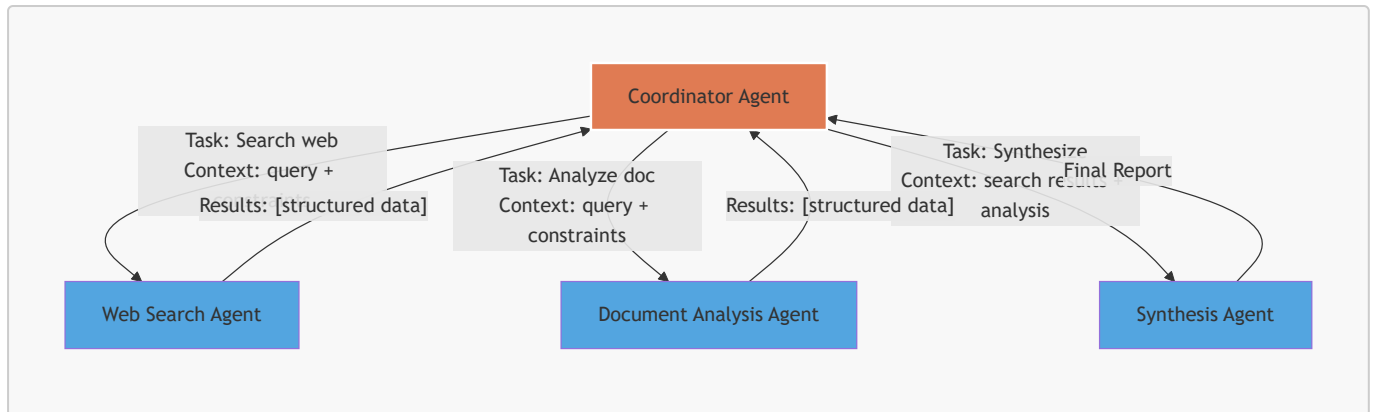
Answer

Coordinator manages all communication:

- Coordinator agent handles all inter-subagent communication, error handling, and information routing
- Subagents operate with isolated context—they don't inherit coordinator's conversation history automatically
- Coordinator decomposes tasks, delegates, aggregates results, and selects which subagents to invoke

Key Concept: Context Passing

Include complete findings from prior agents directly in the subagent's prompt. Use structured data formats to separate content from metadata (source URLs, document names, page numbers).



Card 1.4: Parallel Subagent Execution

Question

How do you spawn parallel subagents?

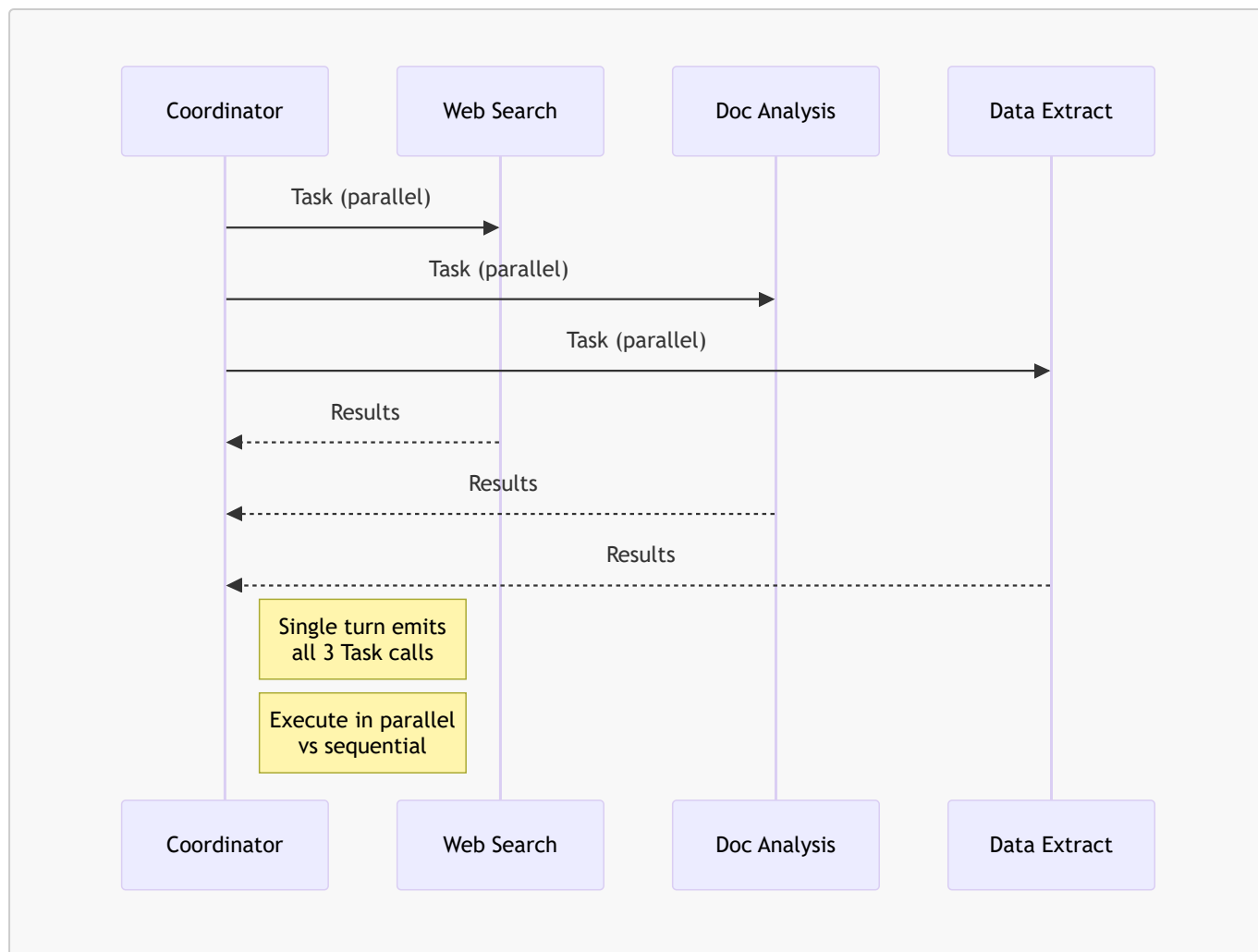
Answer

Emit multiple Task tool calls in a single coordinator response:

- Include "Task" in allowedTools for the coordinator
- Spawn parallel subagents by emitting multiple Task calls in one response rather than across separate turns
- Each subagent receives explicit context in its prompt

Example

When researching a topic, spawn web search, document analysis, and data extraction agents simultaneously from a single coordinator turn.



Card 1.5: Task Decomposition Patterns

Question

What's the difference between prompt chaining and dynamic decomposition?

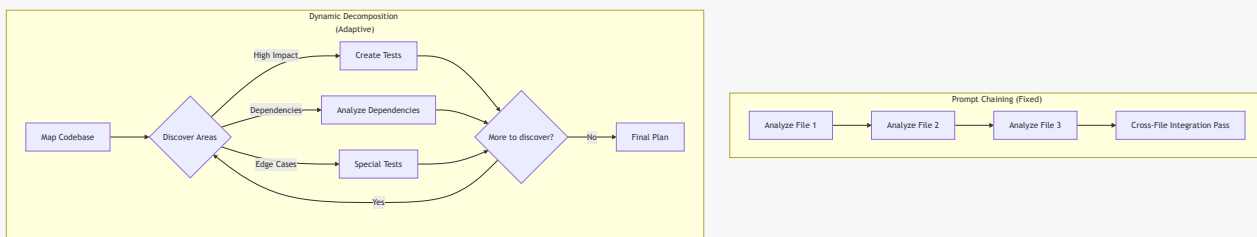
Answer

Use appropriate pattern for the workflow:

- **Prompt chaining:** Fixed sequential pipelines for predictable multi-aspect reviews (e.g., analyze each file individually, then cross-file integration pass)
- **Dynamic decomposition:** Adaptive plans that generate subtasks based on intermediate findings for open-ended investigation

Key Concept

Code review: Use prompt chaining (per-file analysis → cross-file integration). **Legacy codebase testing:** Use dynamic decomposition (map structure → identify high-impact areas → create prioritized adaptive plan).



Card 1.6: Session Management

Question

How do you manage session state and resumption?

Answer

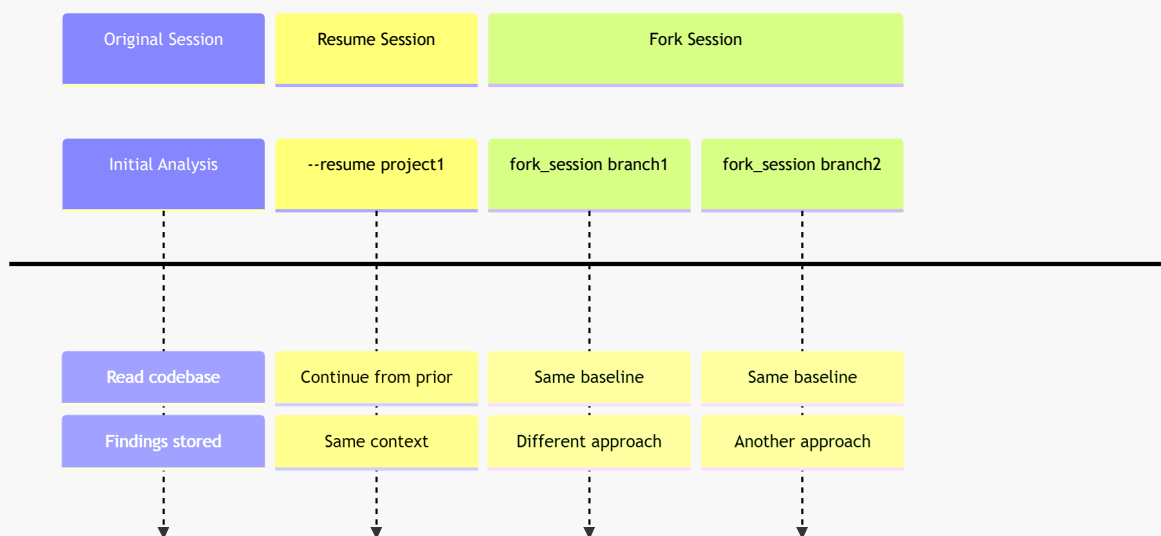
Session management strategies:

- Use `--resume <session-name>` to continue specific prior conversations
- Use `fork_session` to create independent branches from shared analysis baseline
- Start fresh with injected summaries when prior tool results are stale rather than resuming with stale context

Best Practice

Inform a resumed session about specific file changes for targeted re-analysis rather than requiring full re-exploration.

Session Management Options



Card 1.7: Multi-Concern Requests

Question

What's the most reliable way to handle multi-concern customer requests?

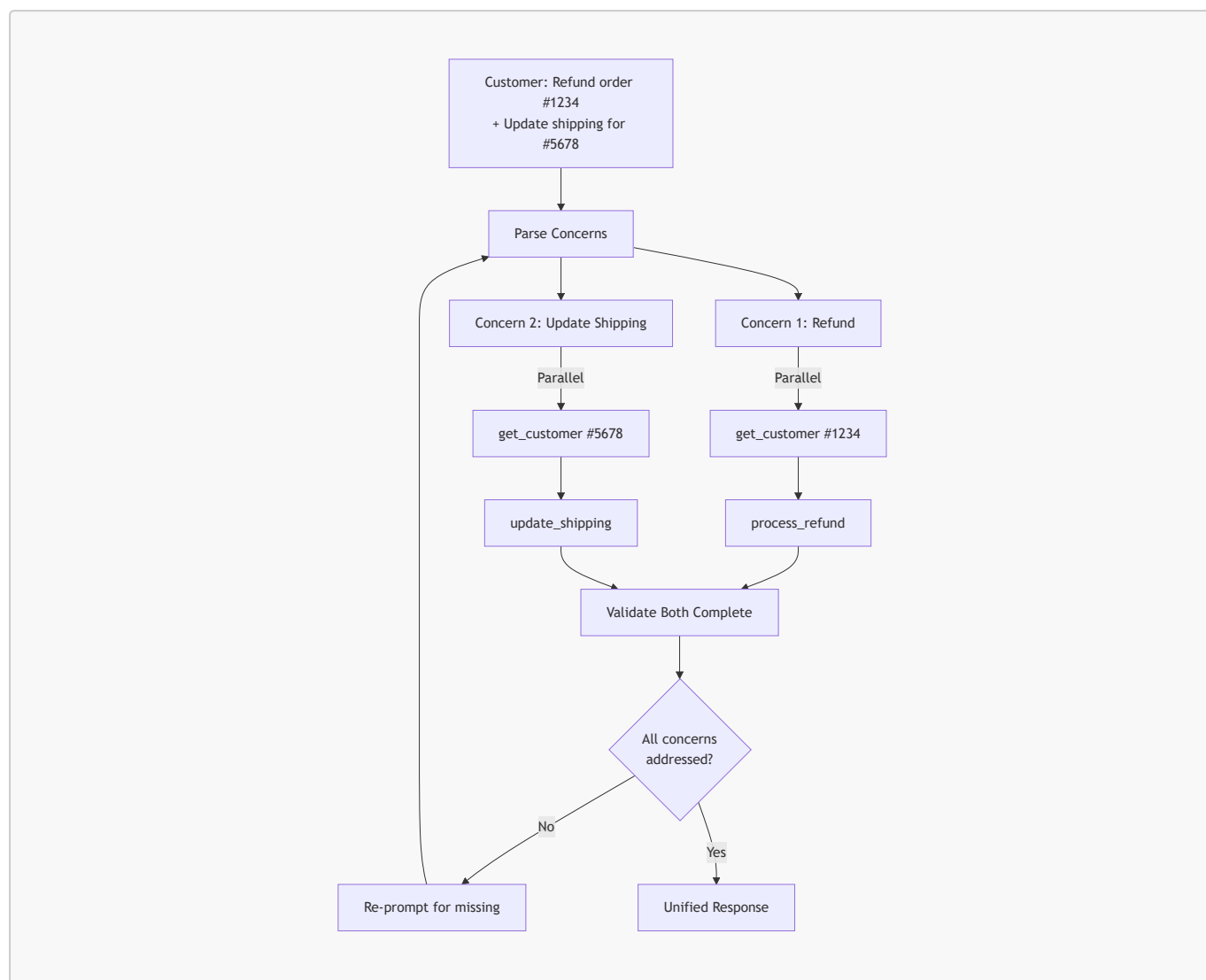
Answer

Decompose and handle in parallel:

- Decompose multi-concern requests into distinct items
- Investigate each in parallel using shared context
- Synthesize unified resolution

Key Concept

Response validation detects incomplete responses and automatically re-prompts the agent to address missed concerns. This is more effective than preprocessing layers or few-shot examples alone.



Card 1.8: Handoff Summaries

Question

How should you structure handoff summaries for escalation?

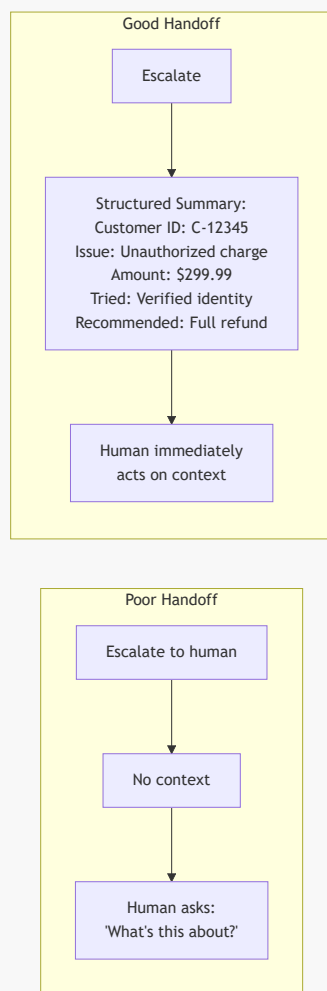
Answer

Include structured context for human agents:

- Customer ID and relevant identifiers
- Root cause analysis
- Refund amounts or relevant values
- Recommended actions
- What has already been attempted

Why Structured?

Human agents may lack access to the conversation transcript. Structured handoff summaries enable them to take over effectively.



Card 1.9: Task Decomposition Risks

Question

What's the risk of overly narrow task decomposition by the coordinator?

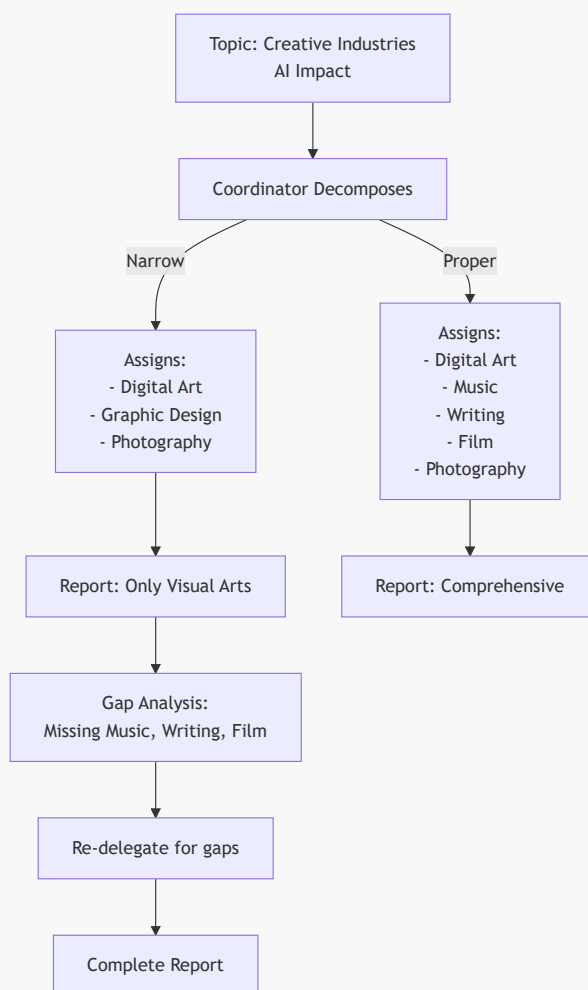
Answer

Incomplete coverage of broad topics:

- Coordinator may decompose "creative industries" into only visual arts (digital art, graphic design, photography)
- Completely misses music, writing, and film
- Subagents execute correctly—the problem is what they were assigned

Solution

Implement iterative refinement loops where coordinator evaluates synthesis output for gaps and re-delegates with targeted queries until coverage is sufficient.



Domain 2: Tool Design & MCP Integration (18%)

Card 2.1: Effective Tool Descriptions

Question

What makes an effective tool description?

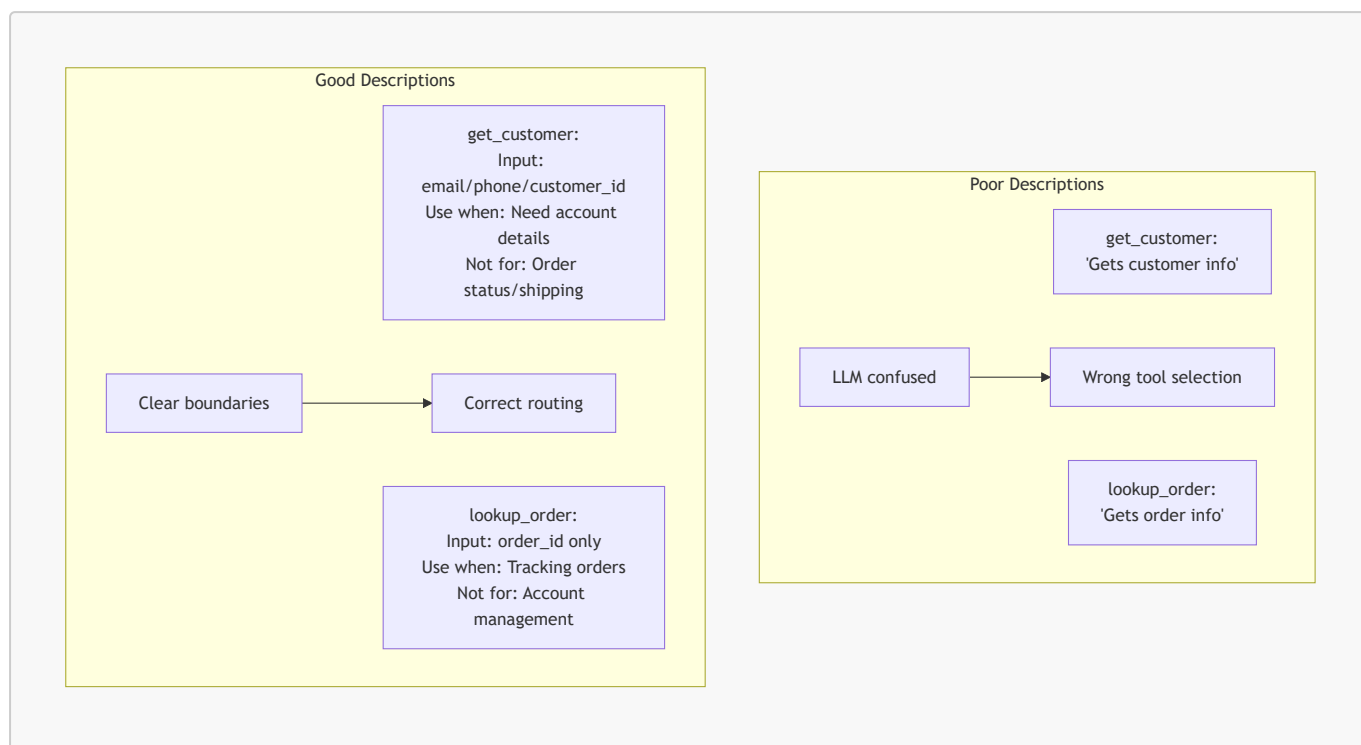
Answer

Clear differentiation and boundaries:

- Expected input formats and example queries
- Edge cases and boundary explanations
- When to use this tool versus similar alternatives

Common Mistake

Minimal descriptions like "Retrieves customer information" lead to unreliable selection. Ambiguous descriptions cause misrouting (e.g., `analyze_content` vs `analyze_document` with near-identical descriptions).



Card 2.2: Structured Error Responses

Question

How do you handle structured error responses in MCP tools?

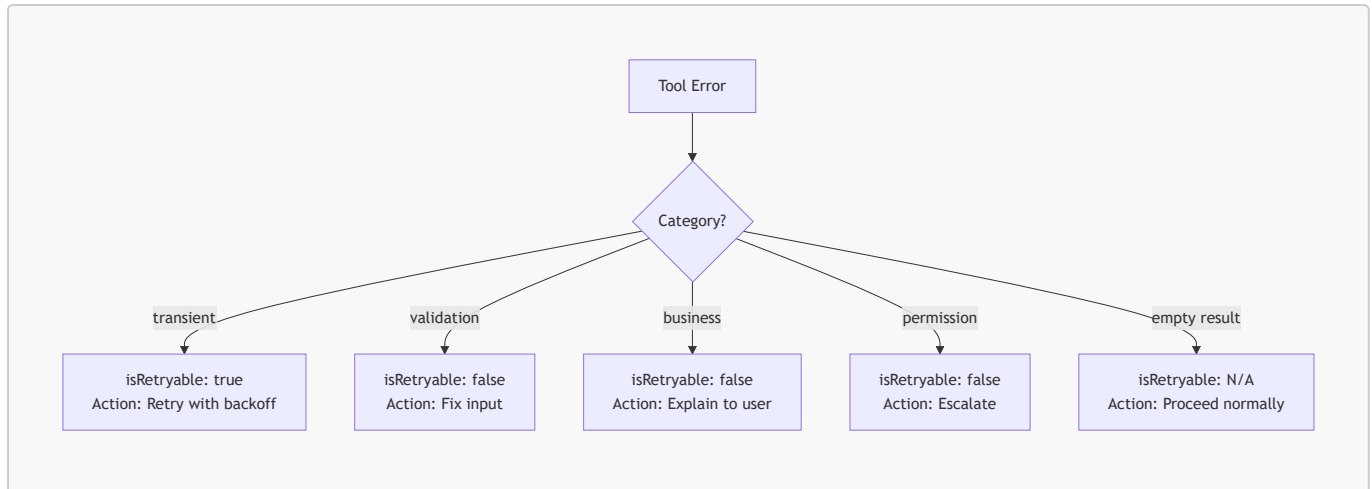
Answer

Return structured error metadata:

- **errorCategory:** transient / validation / permission / business
- **isRetryable:** boolean flag
- Human-readable descriptions
- Partial results and what was attempted

Why It Matters

Generic "Operation failed" responses prevent the agent from making appropriate recovery decisions. Distinguish access failures (need retry) from valid empty results (successful query with no matches).



Card 2.3: Optimal Tool Count

Question

What's the optimal number of tools per agent?

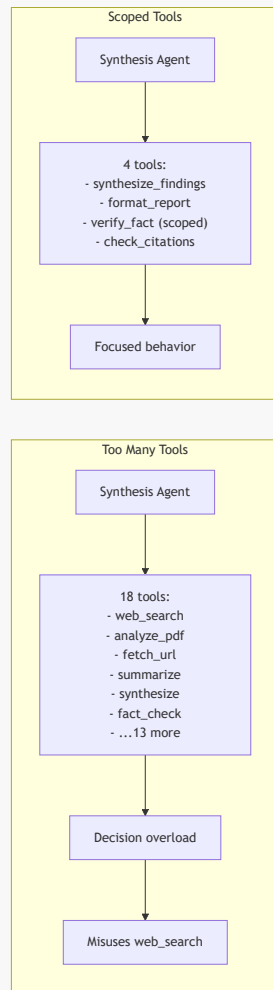
Answer

Limit to 4-5 relevant tools per agent:

- Too many tools (e.g., 18 instead of 4-5) degrades selection reliability by increasing decision complexity
- Agents with tools outside their specialization tend to misuse them
- Provide scoped cross-role tools for high-frequency needs

Example

Synthesis agent shouldn't have web search tools, but can have a scoped `verify_fact` tool for simple lookups.



Card 2.4: Tool Choice Configuration

Question

What are the tool_choice configuration options?

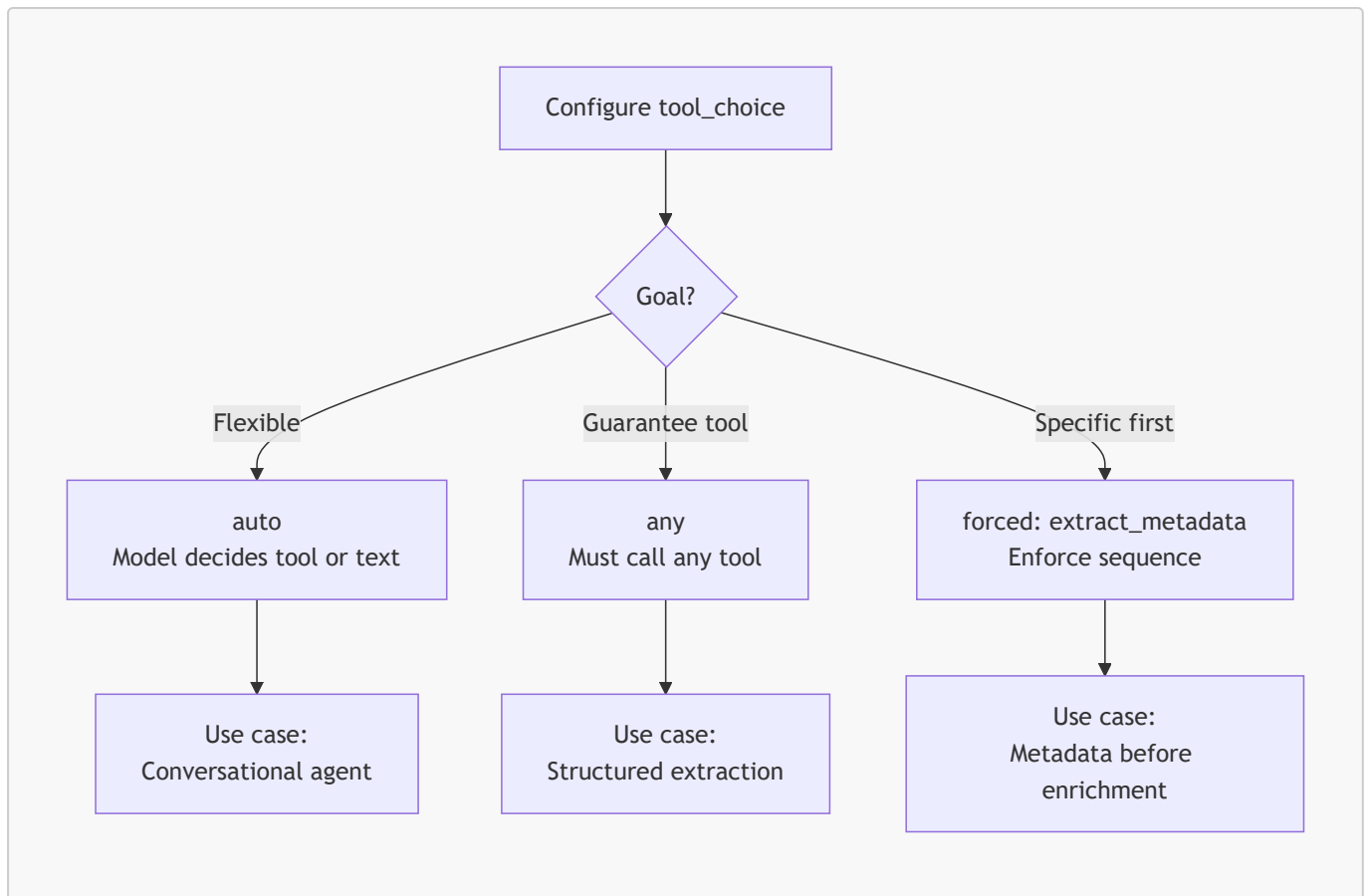
Answer

Control tool selection behavior:

- "auto": Model may return text instead of calling a tool
- "any": Model must call a tool but can choose which (guarantees structured output)
- {"type": "tool", "name": "..."}: Force specific tool selection

Use Cases

Use "any" to guarantee structured output. Use forced selection to ensure metadata extraction runs before enrichment steps.



Card 2.5: MCP Server Scoping

Question

Where should MCP servers be configured?

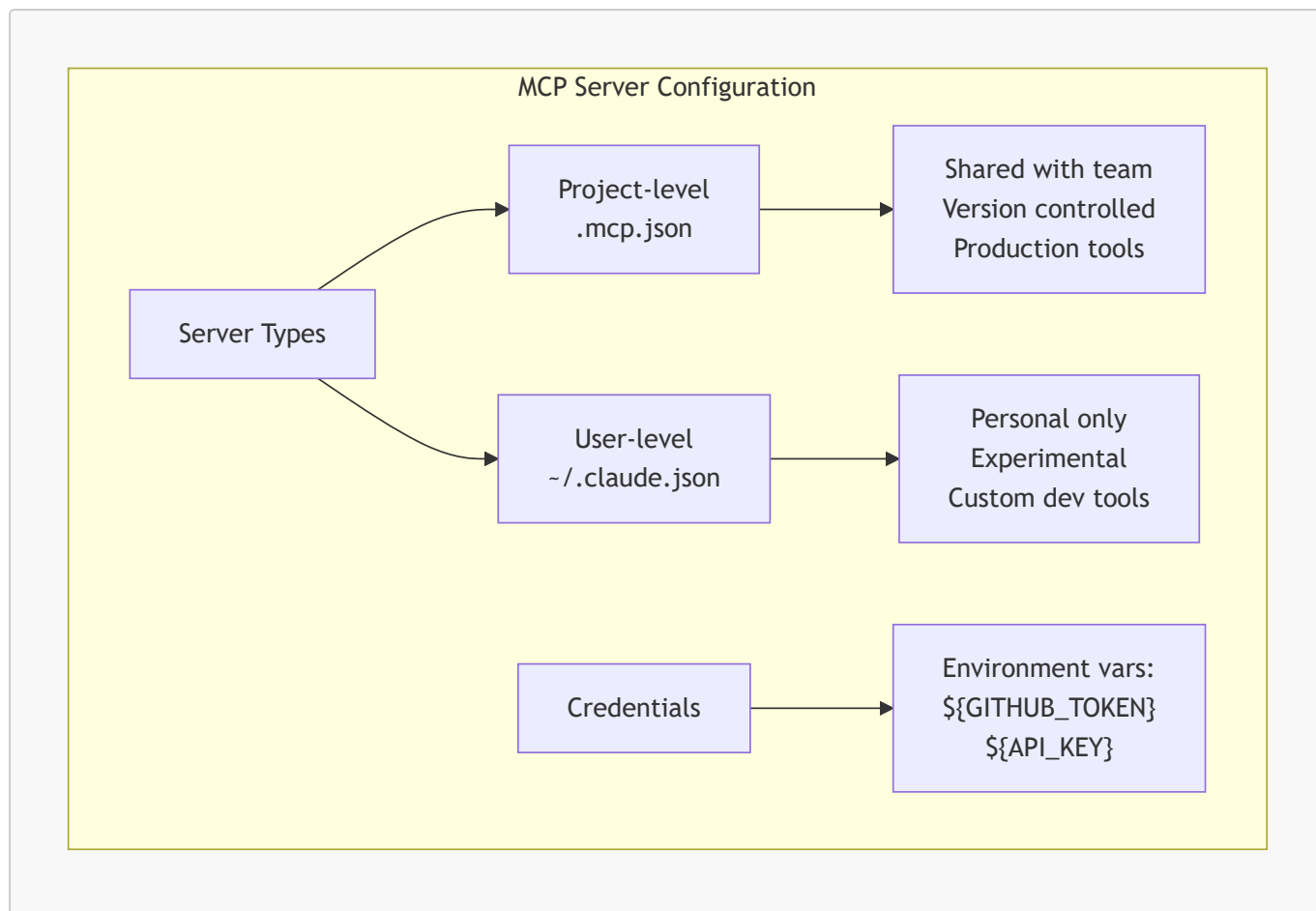
Answer

Scope appropriately:

- **Project-level (.mcp.json):** Shared team tooling, version-controlled
- **User-level (~/.claude.json):** Personal/experimental servers
- Environment variable expansion for credentials: `${GITHUB_TOKEN}`

Best Practice

Choose existing community MCP servers over custom implementations for standard integrations (e.g., Jira). Reserve custom servers for team-specific workflows.



Card 2.6: MCP Resources vs Tools

Question

When should you use MCP resources vs tools?

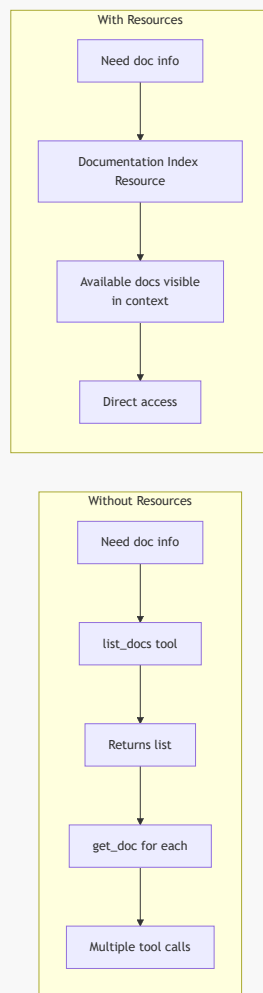
Answer

Different purposes:

- **Resources:** Expose content catalogs (issue summaries, documentation hierarchies, database schemas) to reduce exploratory tool calls
- **Tools:** Actions that perform operations

Resource Example

Expose a "documentation_index" resource so the agent can see available docs without calling multiple list/search tools.



Card 2.7: Built-in Tool Selection

Question

How do you select between built-in tools (Read, Write, Edit, Bash, Grep, Glob)?

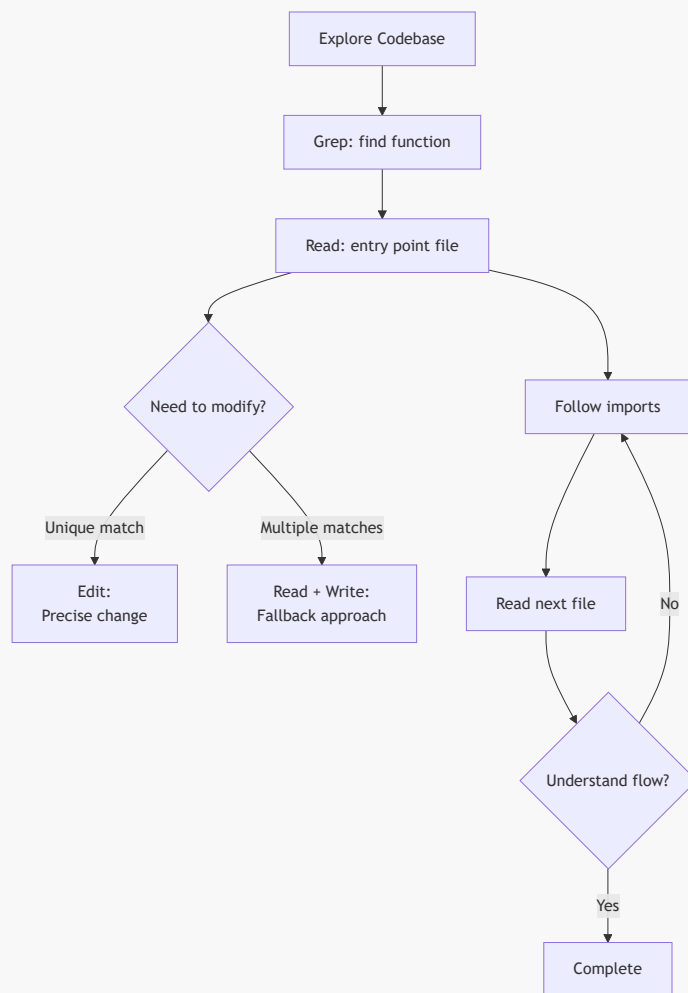
Answer

Use the right tool for the task:

- **Grep:** Search file contents for patterns (function names, error messages, imports)
- **Glob:** Find files by name/extension patterns (`**/*.test.tsx`)
- **Edit:** Targeted modifications with unique text matching
- **Read + Write:** Fallback when Edit can't find unique anchor text

Exploration Pattern

Start with Grep to find entry points, then Read to follow imports and trace flows—rather than reading all files upfront.



Domain 3: Claude Code Configuration & Workflows (20%)

Card 3.1: CLAUDE.md Hierarchy

Question

What's the CLAUDE.md configuration hierarchy?

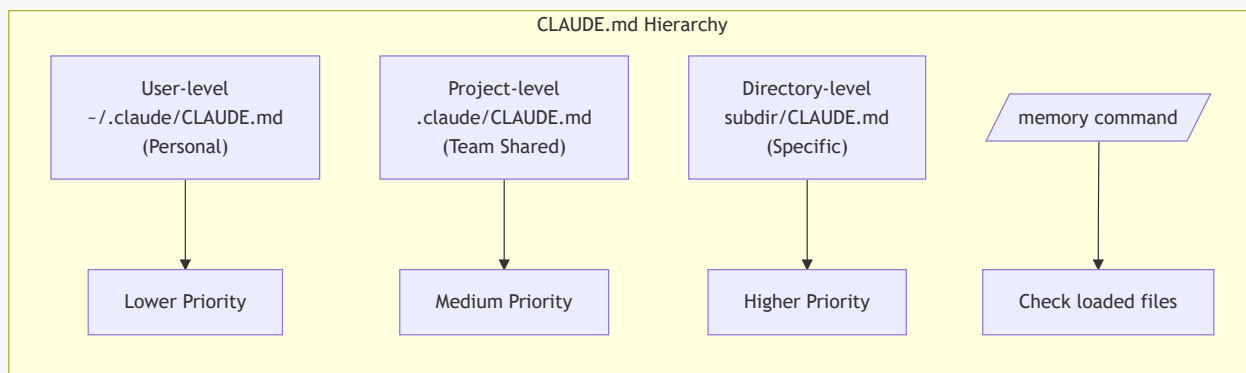
Answer

Configuration levels (highest priority last):

- **User-level:** `~/ .claude/CLAUDE.md` — personal preferences, not shared
- **Project-level:** `.claude/CLAUDE.md` or root `CLAUDE.md` — team standards via version control
- **Directory-level:** Subdirectory `CLAUDE.md` files — specific conventions

Diagnostic

Use `/memory` command to verify which memory files are loaded and diagnose inconsistent behavior.



Card 3.2: Topic-Specific Rules

Question

How do you organize topic-specific rules?

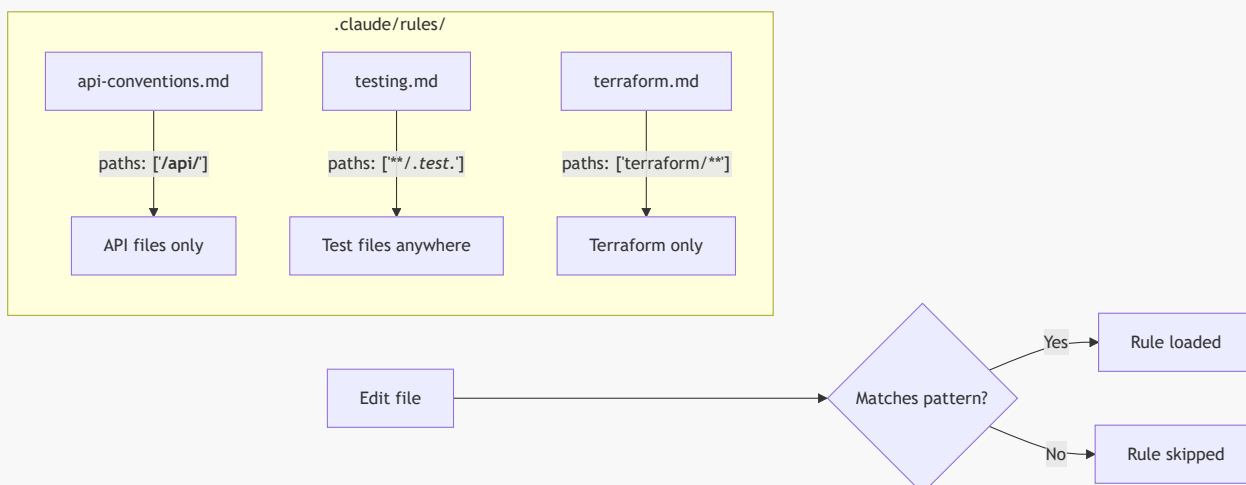
Answer

Use `.claude/rules/` directory:

- Files with YAML frontmatter containing glob patterns for conditional activation
- `paths: ["terraform/**/*"]` — loads only when editing matching files
- Alternative to monolithic CLAUDE.md

Benefits

Path-scoped rules load only when needed, reducing irrelevant context and token usage. Glob patterns apply conventions to files by type regardless of directory location.



Card 3.3: Slash Command Scope

Question

Where should custom slash commands be created?

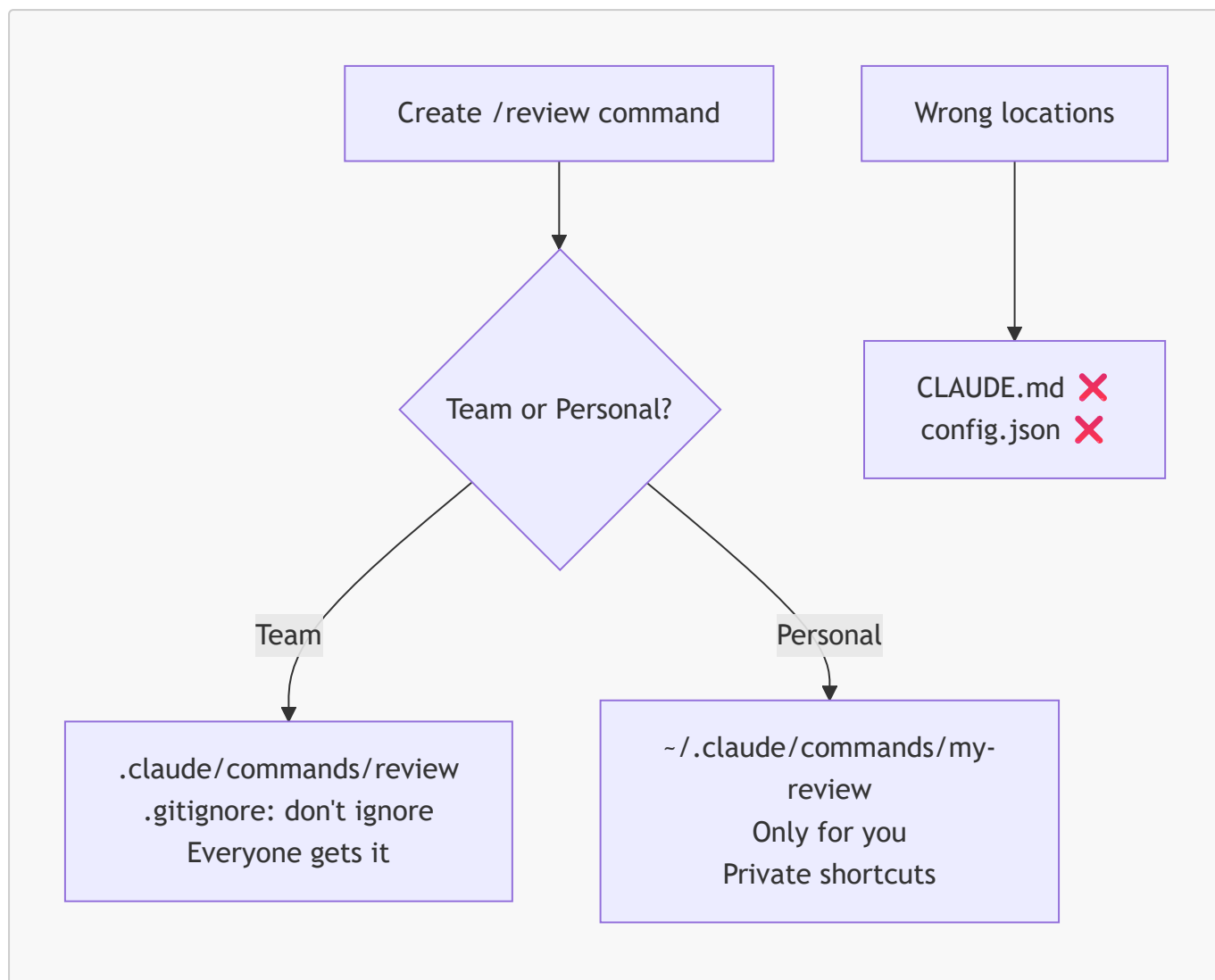
Answer

Scope by sharing needs:

- **Project-scoped:** `.claude/commands/` — shared via version control, available to all developers
- **User-scoped:** `~/.claude/commands/` — personal commands

Note

CLAUDE.md is for project instructions and context, not command definitions. There is no `.claude/config.json` with commands array.



Card 3.4: SKILL.md Frontmatter

Question

What's the SKILL.md frontmatter configuration?

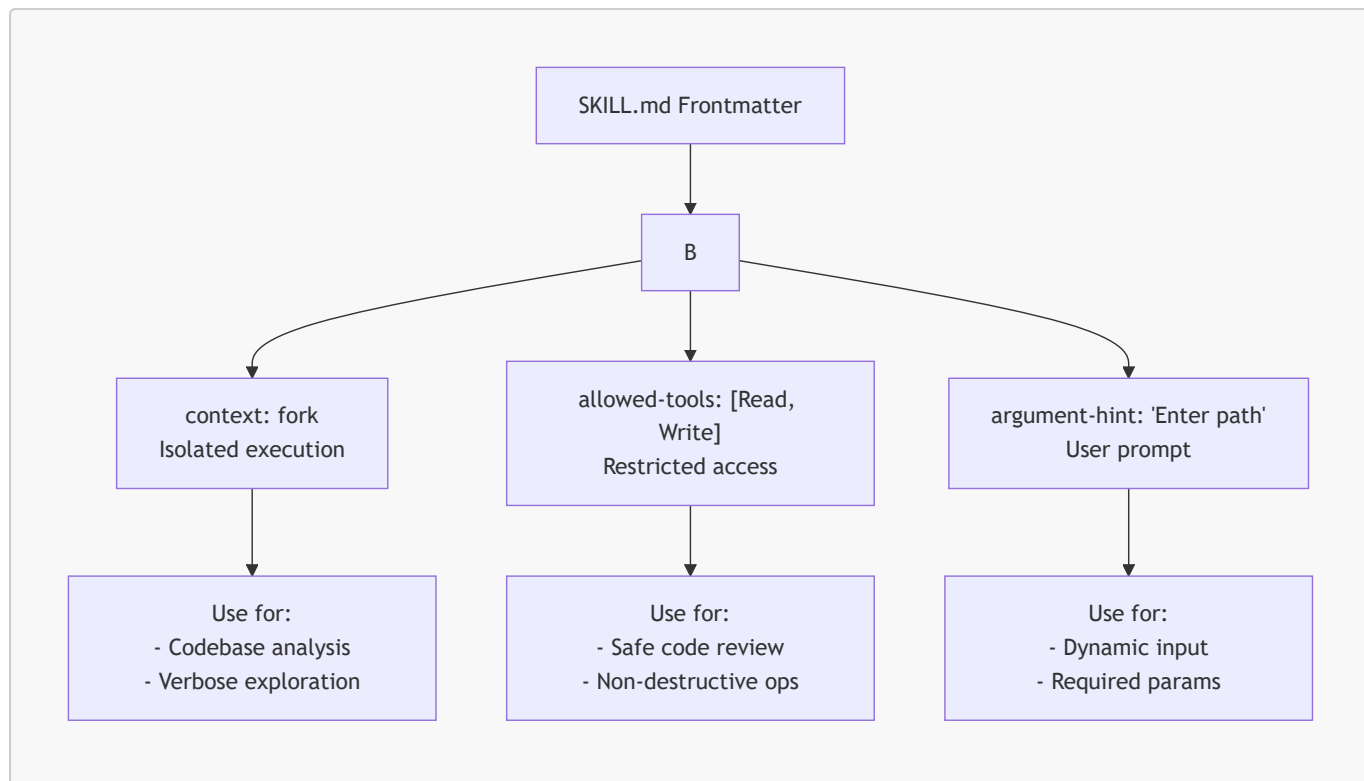
Answer

Available frontmatter options:

- **context: fork** — runs in isolated sub-agent, prevents output pollution
- **allowed-tools** — restricts tool access during execution
- **argument-hint** — prompts for required parameters

Use Cases

Use **context: fork** for skills producing verbose output (codebase analysis) or exploratory context (brainstorming).



Card 3.5: Plan Mode vs Direct Execution

Question

When should you use plan mode vs direct execution?

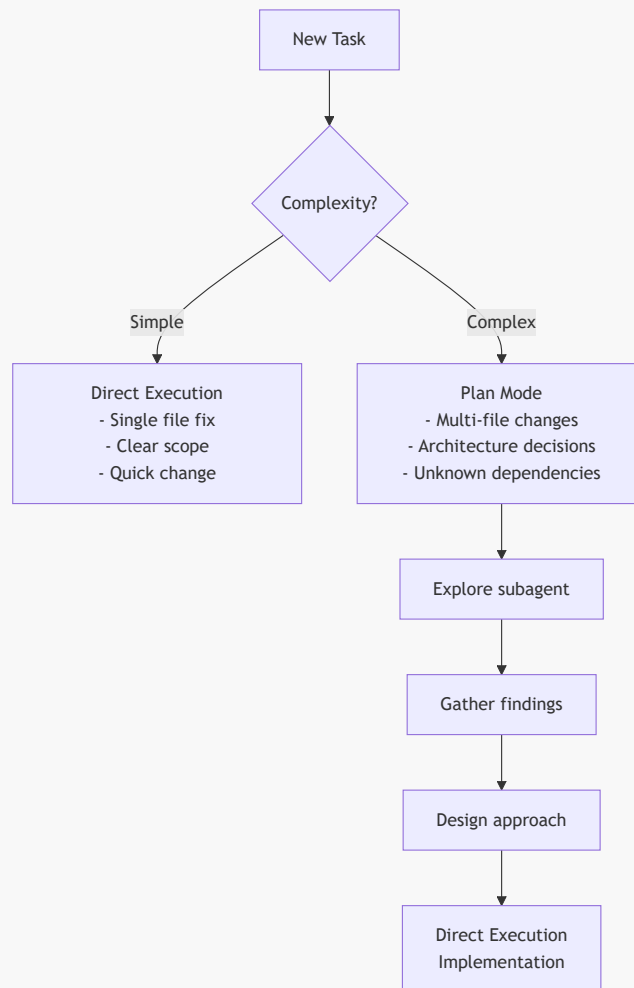
Answer

Match mode to task complexity:

- **Plan mode:** Complex tasks with large-scale changes, architectural decisions, multiple valid approaches, multi-file modifications
- **Direct execution:** Simple, well-scoped changes (single-file bug fix, clear stack trace)

Hybrid Approach

Use plan mode for investigation, then direct execution for implementation (e.g., planning a library migration, then executing the planned approach).



Card 3.6: Explore Subagent

Question

What's the Explore subagent used for?

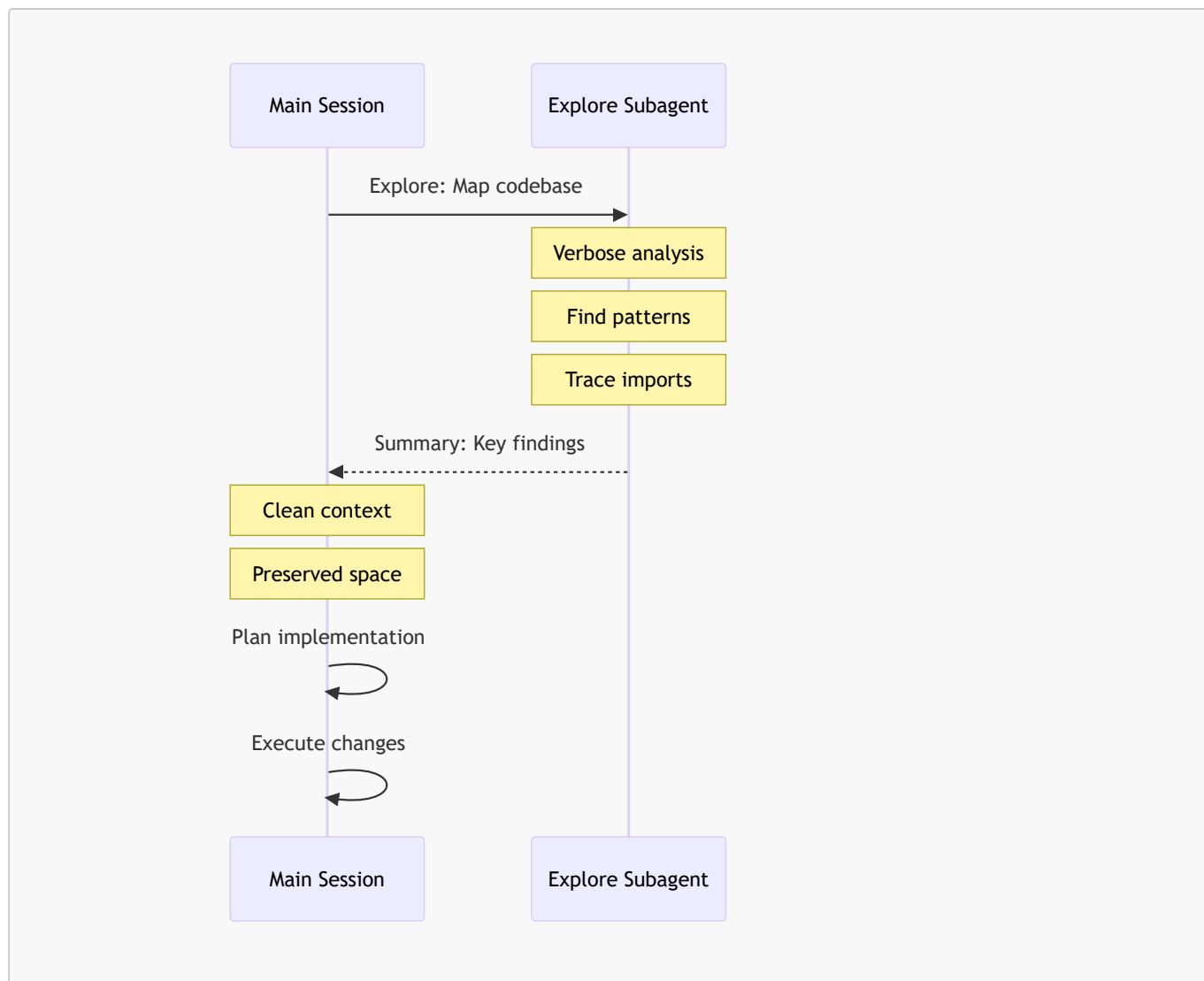
Answer

Isolate verbose discovery output:

- Prevents context window exhaustion during multi-phase tasks
- Returns summaries to preserve main conversation context
- Use during discovery phases before implementation

Workflow

Plan mode → Explore subagent for discovery → Summary returned → Direct execution for implementation.



Card 3.7: Iterative Refinement

Question

What are effective iterative refinement techniques?

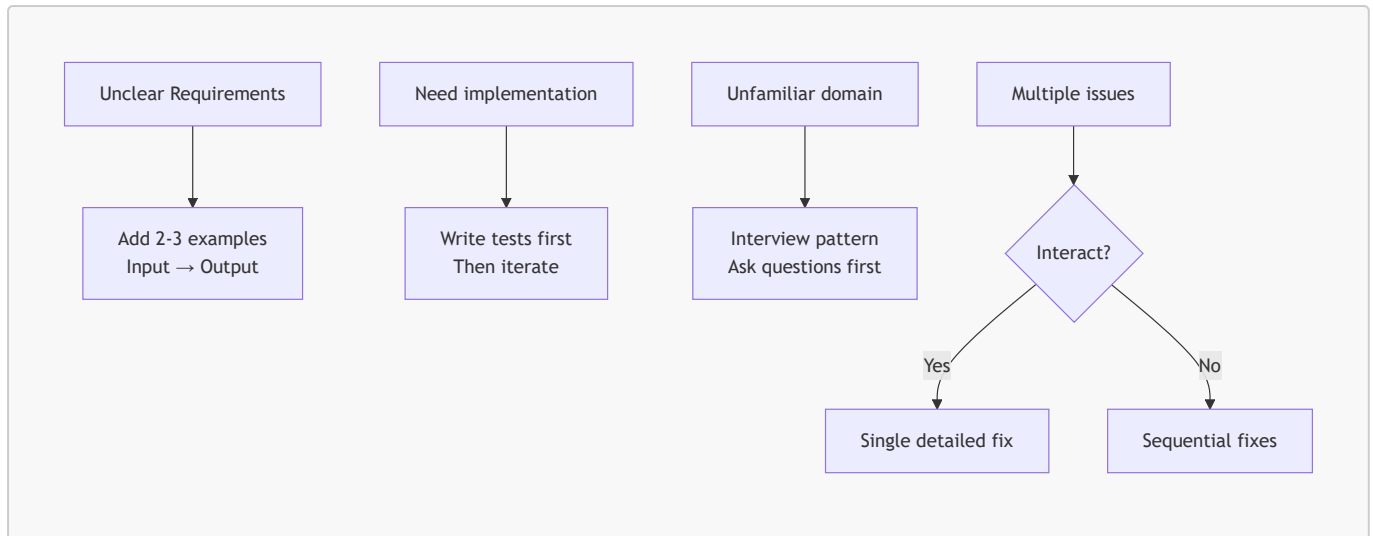
Answer

Progressive improvement strategies:

- Provide 2-3 concrete input/output examples when prose descriptions are interpreted inconsistently
- Test-driven iteration: write test suites first, then iterate by sharing test failures
- Interview pattern: have Claude ask questions to surface considerations before implementing

Issue Management

Address multiple interacting issues in a single detailed message. Fix independent problems sequentially.



Card 3.8: CI/CD Integration

Question

How do you integrate Claude Code into CI/CD pipelines?

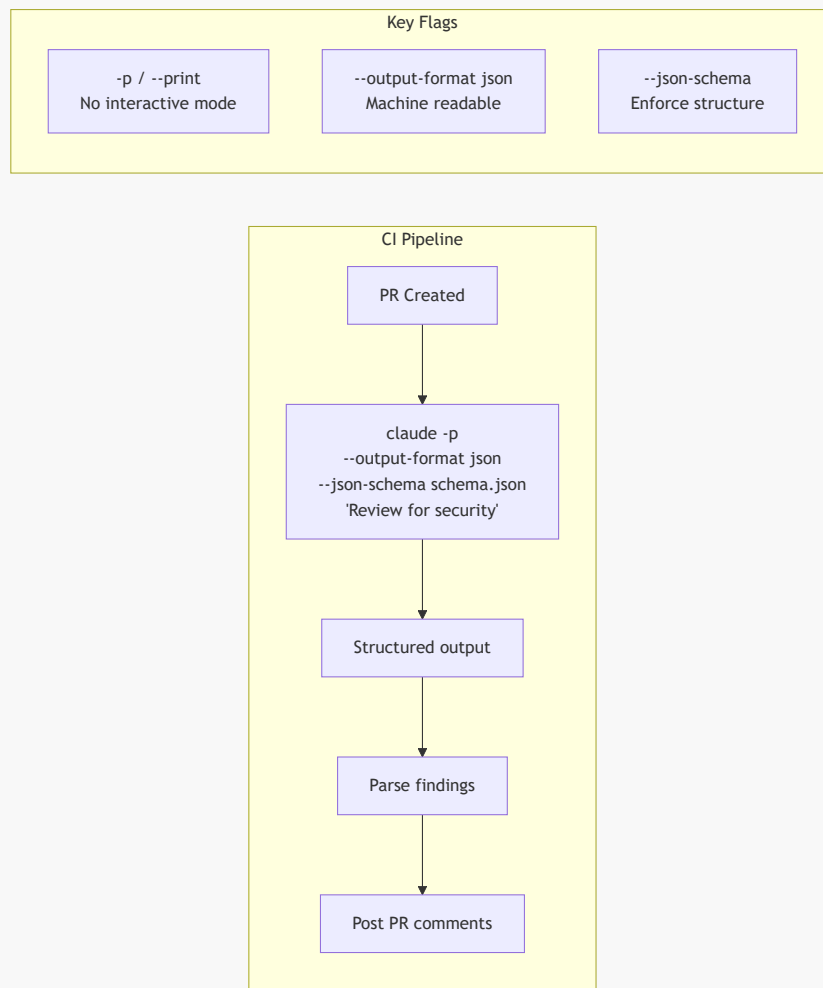
Answer

Non-interactive execution:

- Use `-p` or `--print` flag to run in non-interactive mode
- Use `--output-format json` with `--json-schema` for machine-parseable structured findings
- CLAUDE.md provides project context (testing standards, review criteria) to CI

Session Isolation

Same session that generated code is less effective at reviewing its own changes. Use independent review instance.



Domain 4: Prompt Engineering & Structured Output (20%)

Card 4.1: Explicit Criteria

Question

What's the most effective way to improve prompt precision?

Answer

Explicit criteria over vague instructions:

- "Flag comments only when claimed behavior contradicts actual code behavior" vs "check that comments are accurate"
- Avoid confidence-based filtering like "be conservative" or "only report high-confidence findings"

False Positive Impact

High false positive categories undermine developer trust in accurate categories. Temporarily disable high false-positive categories to restore trust while improving prompts.



Card 4.2: Few-Shot Prompting

Question

How do few-shot examples improve output consistency?

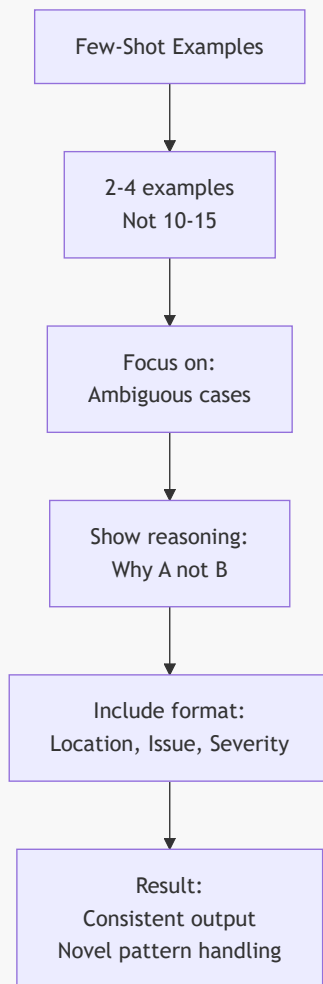
Answer

Most effective technique for consistent formatting:

- 2-4 targeted examples for ambiguous scenarios showing reasoning for choices
- Demonstrate specific desired output format (location, issue, severity, suggested fix)
- Show acceptable patterns vs genuine issues to reduce false positives while enabling generalization

Key Benefit

Few-shot examples enable the model to generalize judgment to novel patterns rather than matching only pre-specified cases.



Card 4.3: Structured Output with Tool Use

Question

What's the most reliable approach for guaranteed structured output?

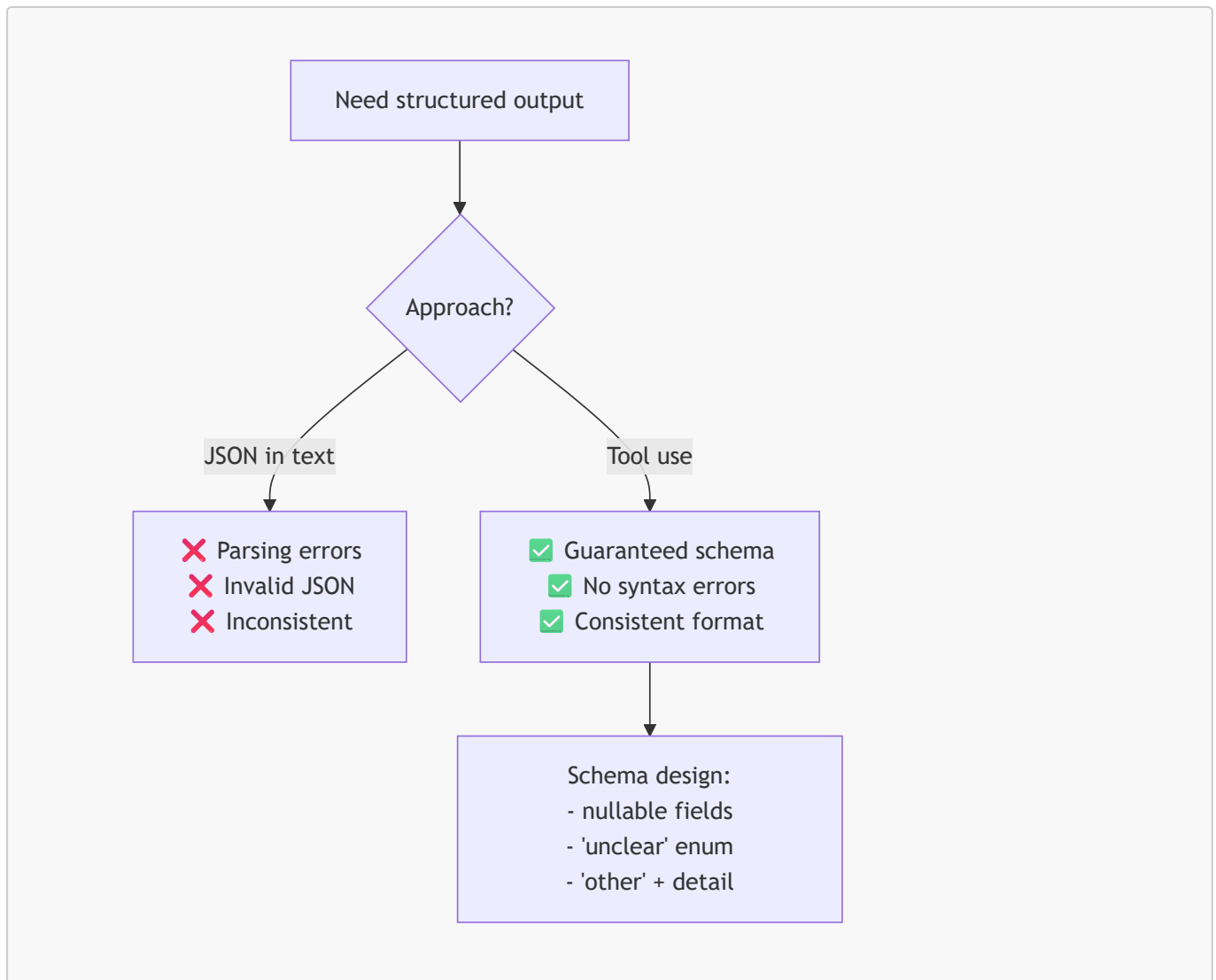
Answer

Tool use with JSON schemas:

- Eliminates JSON syntax errors entirely
- Strict schemas don't prevent semantic errors (values in wrong fields, line items not summing to total)

Schema Design

Use optional/nullable fields when source documents may not contain information (prevents fabrication). Add enum values like "unclear" for ambiguous cases and "other" + detail fields for extensible categorization.



Card 4.4: Validation-Retry Loops

Question

How do you implement validation-retry loops?

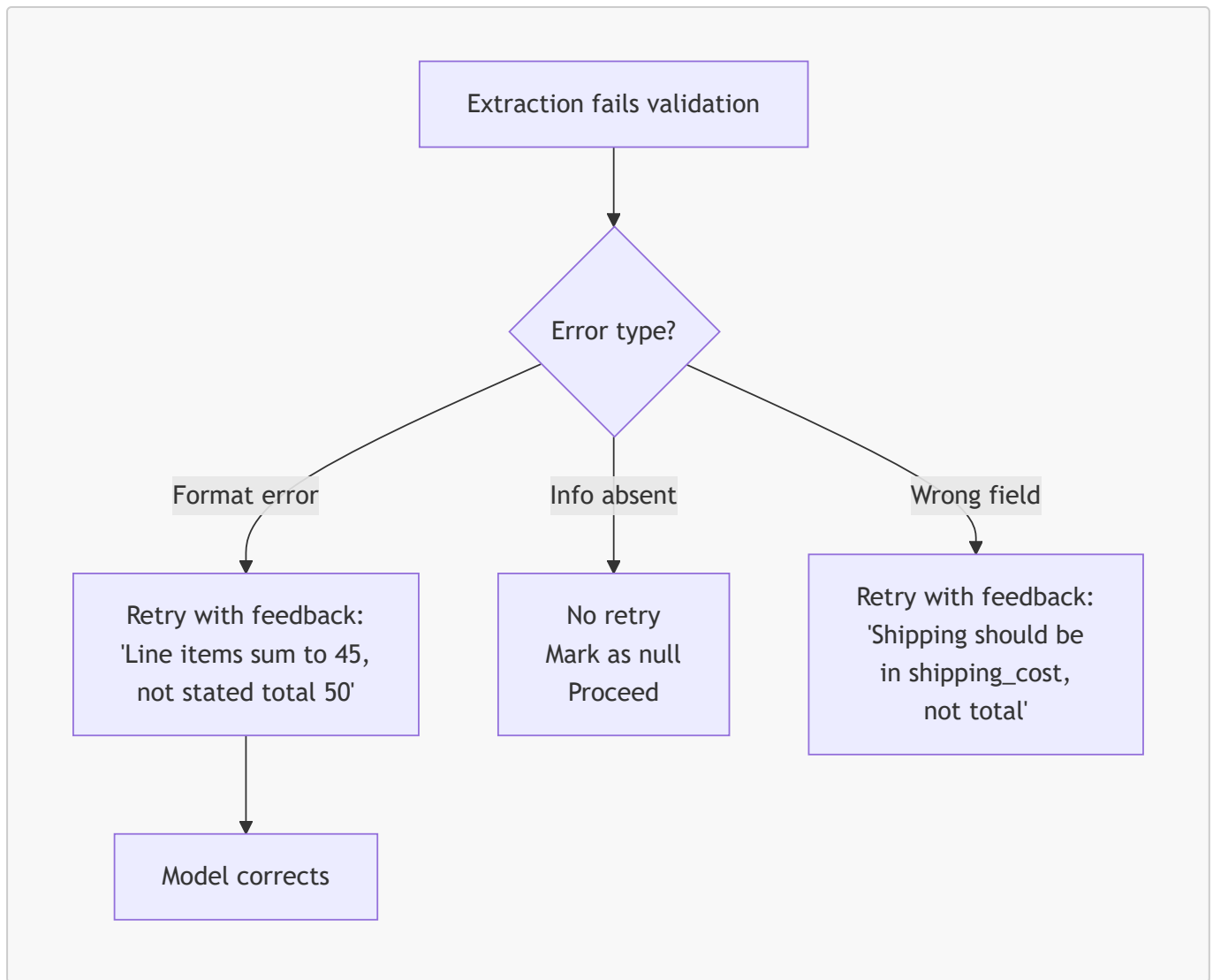
Answer

Retry with specific error feedback:

- Append specific validation errors to the prompt on retry to guide correction
- Send follow-up with original document, failed extraction, and specific validation errors
- Track which errors are resolvable (format mismatches) vs not (information absent)

When Retries Fail

Retries are ineffective when required information is simply absent from the source document vs format/structural errors.



Card 4.5: Message Batches API

Question

When is the Message Batches API appropriate?

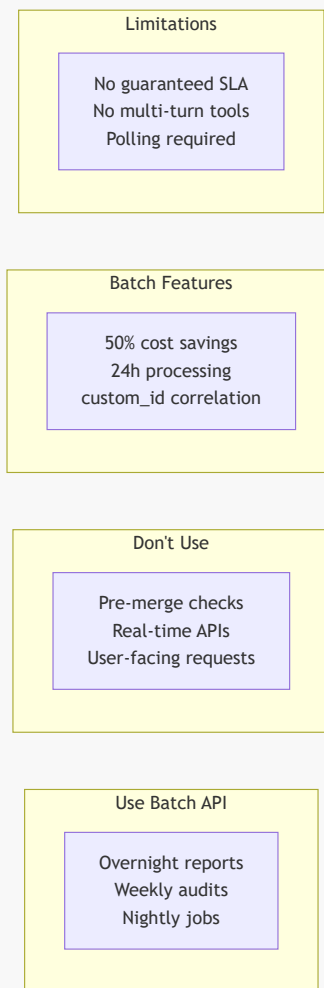
Answer

50% cost savings for latency-tolerant workloads:

- **Appropriate:** Overnight reports, weekly audits, nightly test generation
- **Inappropriate:** Blocking workflows, pre-merge checks
- Up to 24-hour processing window, no guaranteed latency SLA

Limitations

Batch API does not support multi-turn tool calling within a single request. Use `custom_id` fields to correlate request/response pairs.



Card 4.6: Multi-Pass Review

Question

How do you design multi-pass review architectures?

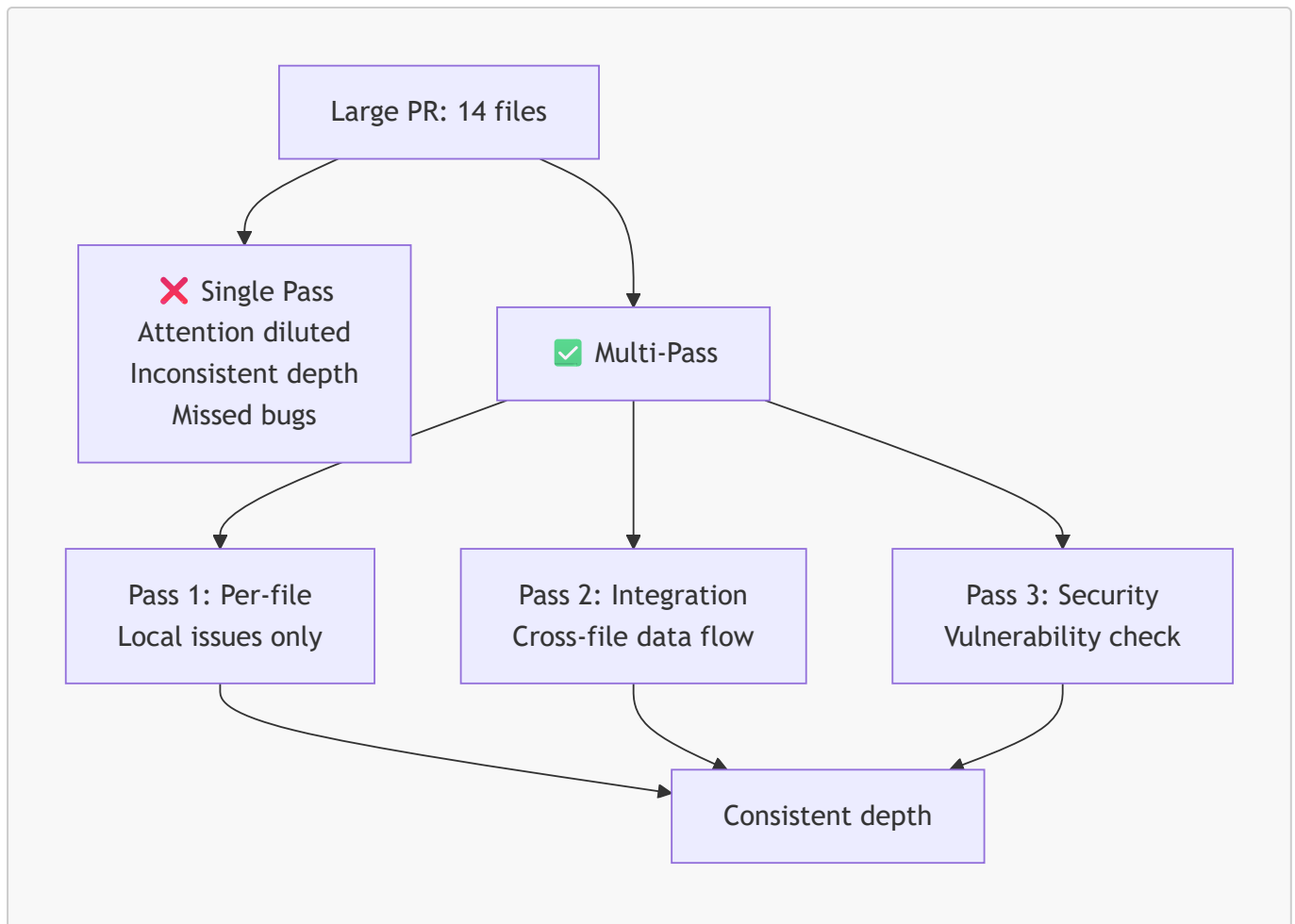
Answer

Split large reviews into focused passes:

- Per-file local analysis passes for local issues
- Separate cross-file integration pass for data flow analysis
- Avoids attention dilution and contradictory findings

Self-Review Limitation

Independent review instances (without prior reasoning context) are more effective at catching subtle issues than self-review instructions or extended thinking.



Domain 5: Context Management & Reliability (15%)

Card 5.1: Progressive Summarization Risks

Question

What are the risks of progressive summarization?

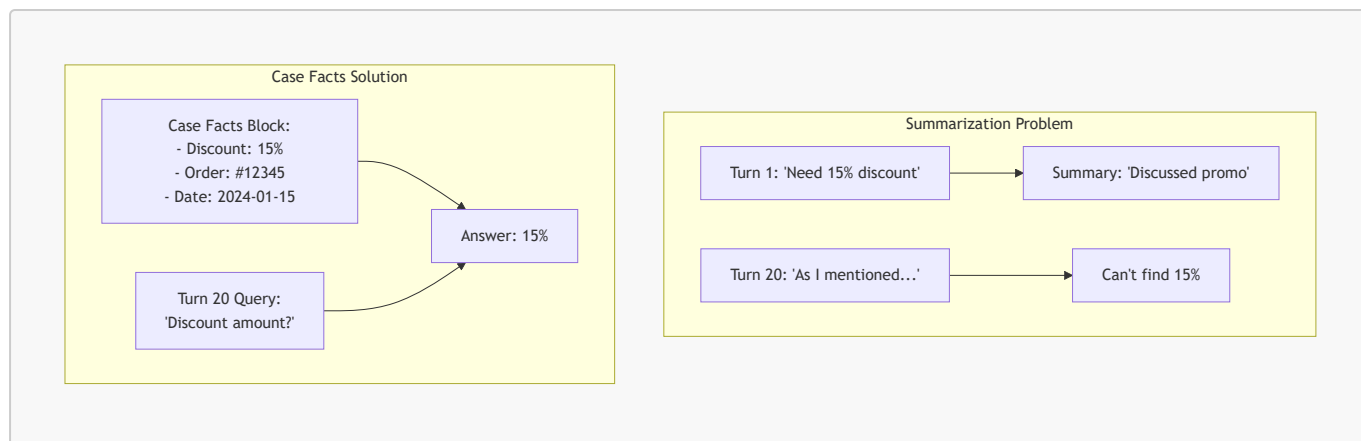
Answer

Fundamental information loss:

- Condensing numerical values, percentages, dates into vague summaries
- "Discussed promotional pricing" loses the specific 15% discount mentioned
- Cannot reliably "prompt your way out" of information loss

Solution

Extract transactional facts (amounts, dates, order numbers) into a persistent "case facts" block included in each prompt, outside summarized history.



Card 5.2: Lost in the Middle Effect

Question

What's the 'lost in the middle' effect?

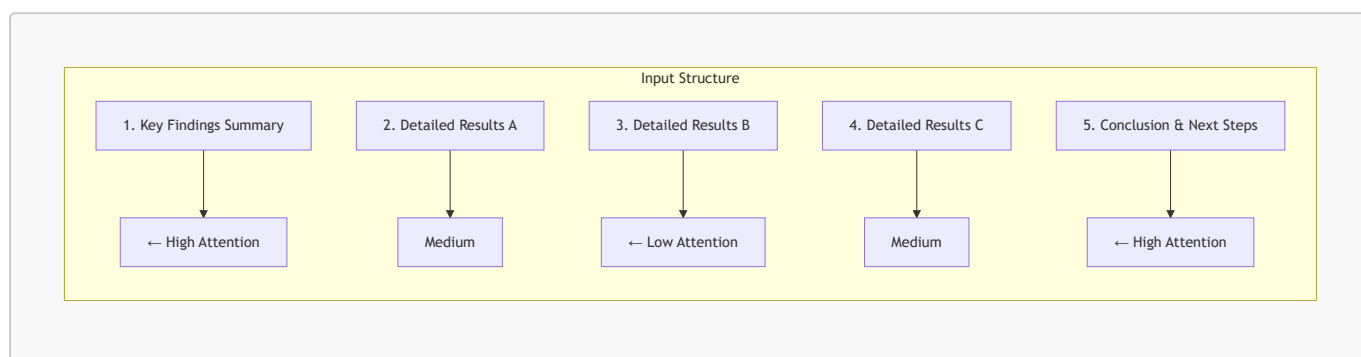
Answer

Position-based attention degradation:

- Models reliably process information at beginning and end of long inputs
- May omit findings from middle sections
- Affects long documents and multi-turn conversations

Mitigation

Place key findings summaries at beginning of aggregated inputs. Organize detailed results with explicit section headers.



Card 5.3: Escalation Triggers

Question

When should you escalate to human agents?

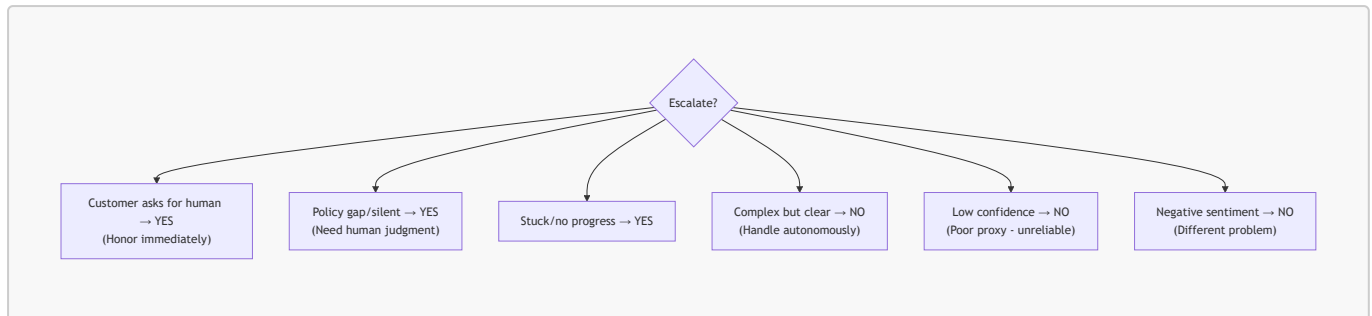
Answer

Appropriate escalation triggers:

- Customer explicitly requests a human (honor immediately)
- Policy exceptions/gaps (not just complex cases)
- Inability to make meaningful progress

Anti-patterns

Sentiment-based escalation and self-reported confidence scores are unreliable proxies for case complexity. Multiple customer matches require clarification (request additional identifiers) rather than heuristic selection.



Card 5.4: Error Propagation

Question

How do you propagate errors in multi-agent systems?

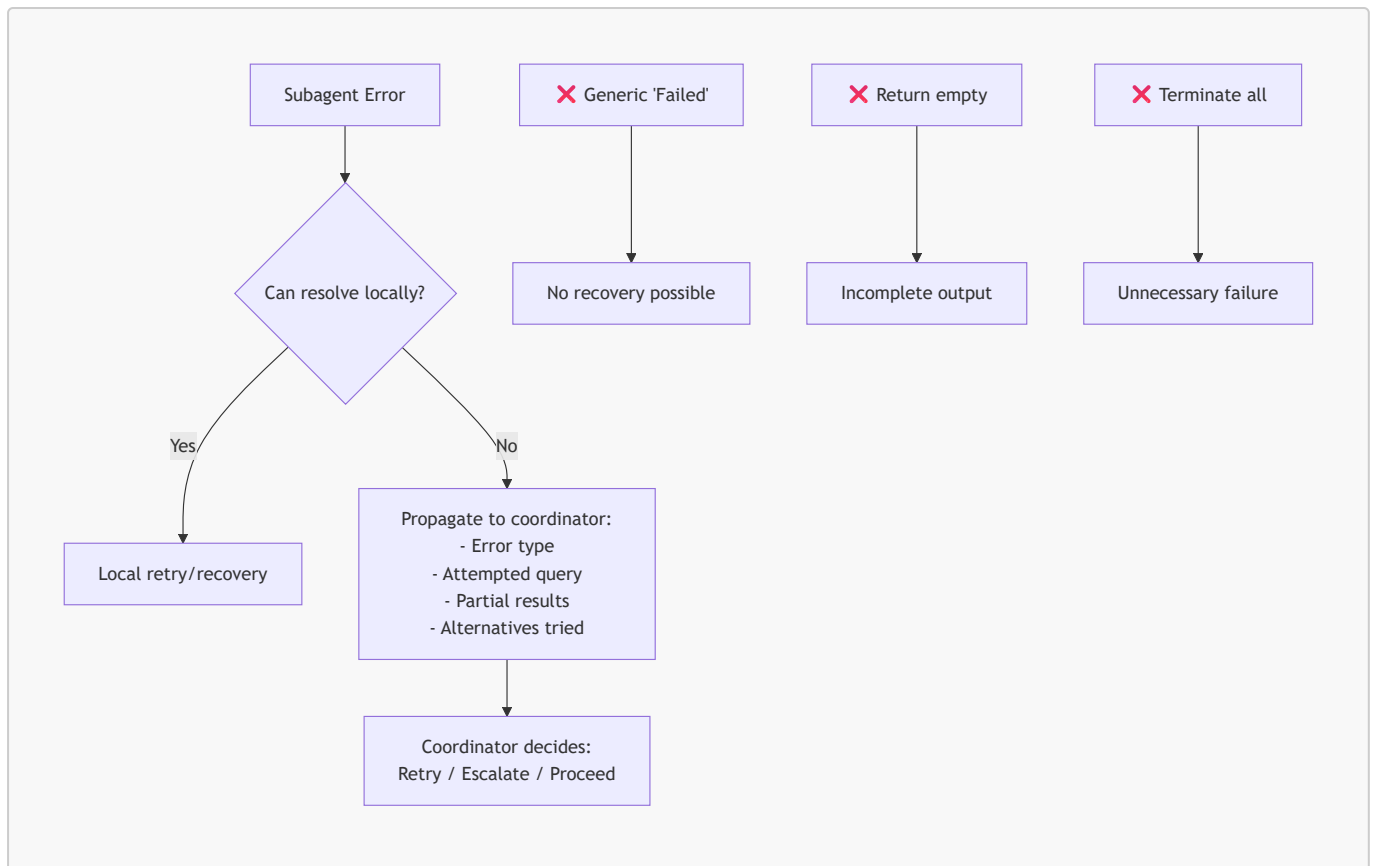
Answer

Structured error context enables recovery:

- Include failure type, attempted query, partial results, alternative approaches
- Distinguish access failures (need retry) from valid empty results
- Implement local recovery within subagents before propagating

Anti-patterns

Avoid generic error statuses ("search unavailable") that hide context. Don't silently suppress errors (return empty as success) or terminate entire workflows on single failures.



Card 5.5: Large Codebase Context

Question

How do you manage context in large codebase exploration?

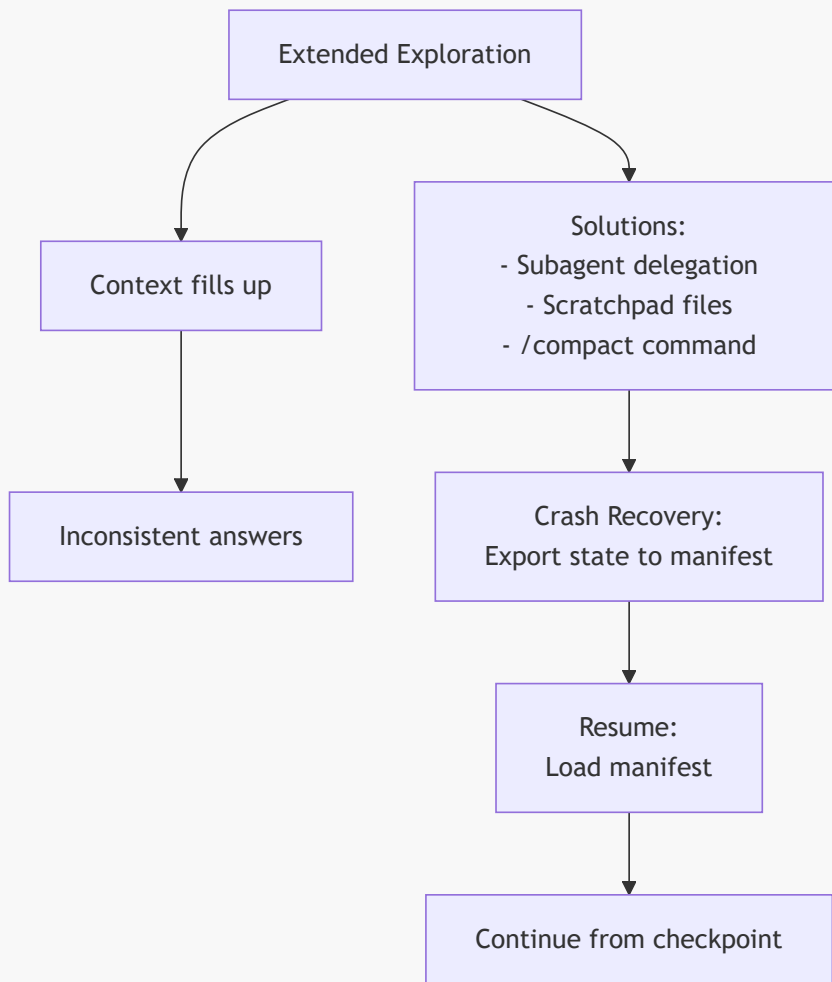
Answer

Combat context degradation:

- Spawn subagents to investigate specific questions while main agent coordinates
- Maintain scratchpad files recording key findings
- Summarize findings before spawning sub-agents for next phase
- Use `/compact` during extended exploration

Crash Recovery

Design structured agent state exports (manifests) that coordinator loads on resume and injects into agent prompts.



Card 5.6: Human Review Workflows

Question

How do you design human review workflows?

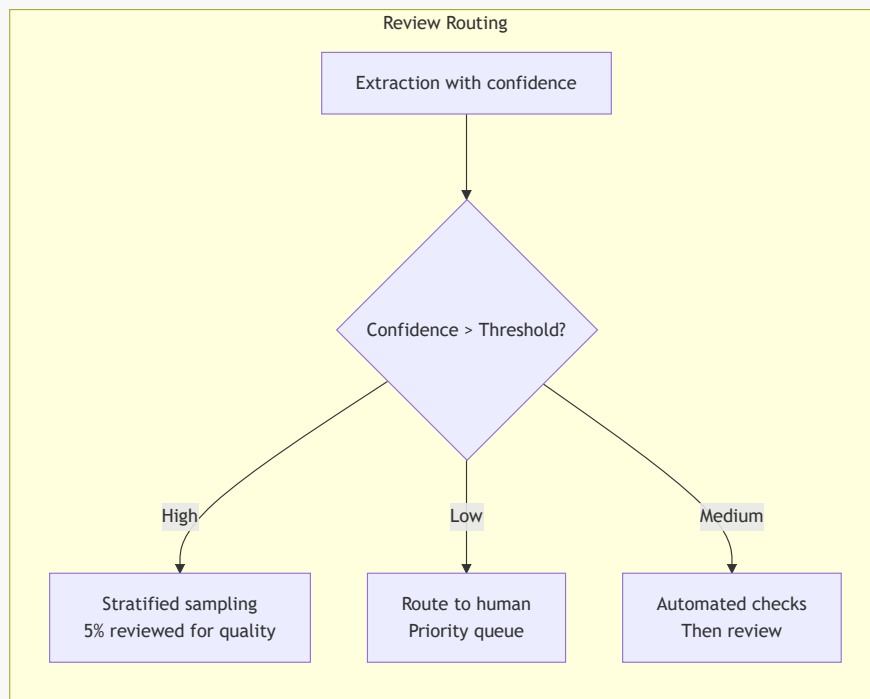
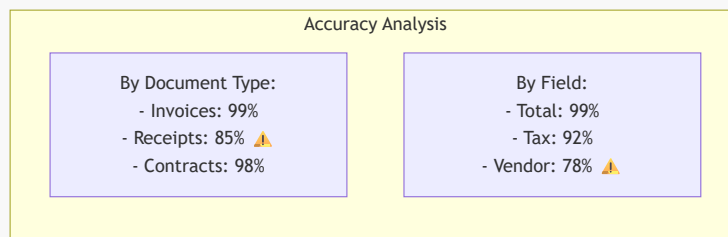
Answer

Calibrated review routing:

- Implement stratified random sampling of high-confidence extractions for ongoing error rate measurement
- Analyze accuracy by document type and field before reducing human review
- Output field-level confidence scores and calibrate thresholds using labeled validation sets

Risk

Aggregate accuracy metrics (97% overall) may mask poor performance on specific document types or fields.



Card 5.7: Information Provenance

Question

How do you preserve information provenance in synthesis?

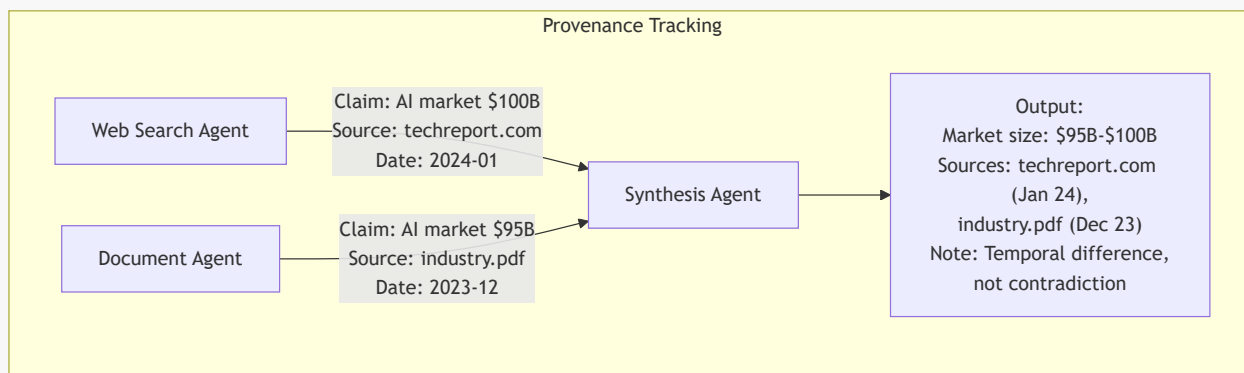
Answer

Maintain claim-source mappings:

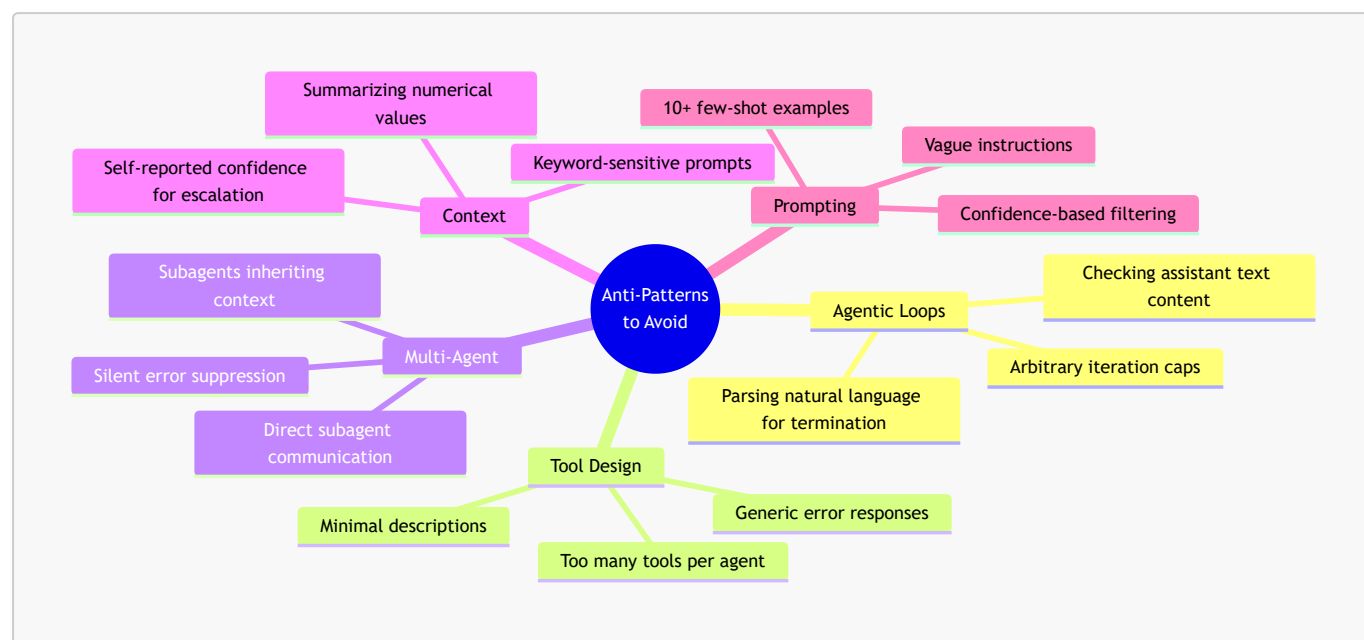
- Require subagents to output structured claim-source mappings (source URLs, document names, excerpts)
- Include publication/collection dates to prevent temporal differences being misinterpreted as contradictions
- Annotate conflicting statistics with source attribution rather than arbitrarily selecting one

Output Structure

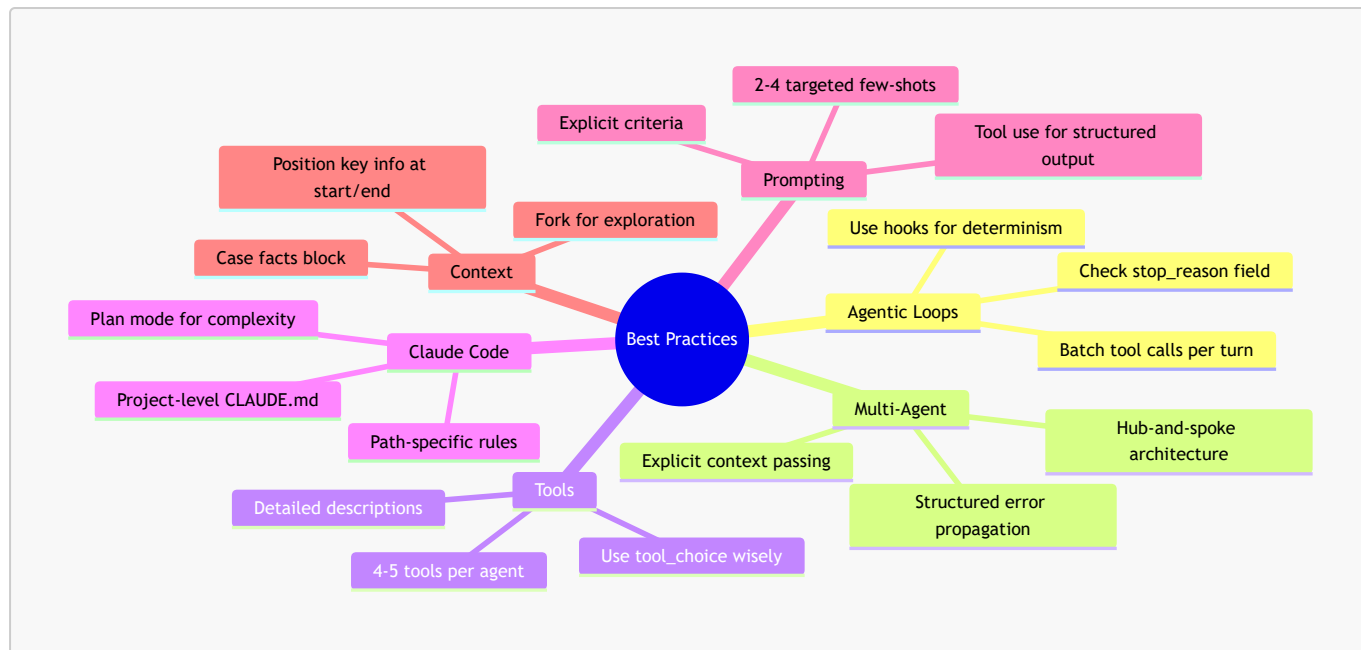
Distinguish well-established findings from contested ones in reports. Preserve original source characterizations and methodological context.



Quick Reference: Anti-Patterns Summary



Quick Reference: Best Practices Summary



Domain Weightings Reference

Domain	Weight	Cards
Domain 1: Agentic Architecture & Orchestration	27%	9 cards
Domain 2: Tool Design & MCP Integration	18%	7 cards
Domain 3: Claude Code Configuration	20%	8 cards
Domain 4: Prompt Engineering	20%	6 cards
Domain 5: Context Management	15%	7 cards

Exam Day Checklist

Before the Exam:

- ☐ Review all 5 domains
- ☐ Study the 12 sample questions in official guide
- ☐ Complete hands-on exercises
- ☐ Know the anti-patterns vs best practices

Key Topics to Master:

- ☐ `stop_reason` handling (`tool_use` vs `end_turn`)
- ☐ Programmatic vs prompt-based enforcement
- ☐ Hub-and-spoke multi-agent architecture
- ☐ Tool description quality
- ☐ MCP server scoping
- ☐ CLAUDE.md hierarchy

- ☐ Plan mode vs direct execution
 - ☐ Tool use for structured output
 - ☐ Context window management
 - ☐ Escalation triggers
-

Good luck on your Claude Certified Architect - Foundations exam!